Did It Leave the Cloth? Detecting Airborne Balls in Pool Video from Overhead and Broadcast Cameras

Working draft — billiard-research

October 23, 2026

Abstract

Jump shots, hops off the break and balls that ride up over one another are routine in pool, but every vision system for the game we know of treats the balls as points on a plane. We ask how well a fixed camera can tell when a ball has left the cloth, and which visual evidence carries that information. Because no billiards dataset labels ball height, we build one: a 3D billiards simulator with an airborne phase (ballistic flight, slate bounces, 3D ball–ball collisions) coupled to a physically based renderer that produces video from an overhead table camera and from an elevated broadcast camera, with exact per-frame height ground truth. A geometric analysis shows that the cue most often proposed for overhead cameras, apparent ball size, is essentially unusable: a ball 1 cm off the cloth grows by 0.05 px at 720p (0.14 px at 4K), while the shadows cast by off-axis lamps move by several pixels per centimetre. We then evaluate a detector built on three pixel-derived cue families — size, shadow and motion (a rolling-versus-ballistic model comparison) — on 60 held-out randomised shots. Using both views, the detector reaches a frame-level average precision (AP) of 0.96 and finds 90% of jump events. With a single camera, the broadcast view (AP 0.93) beats the overhead view (AP 0.86), where the shadow is the best single cue. Flights that rise above 3 cm are found in every view; centimetre hops, and collisions, which cause most false alarms, remain open problems. We release the simulator, renderer, clips and detector, and describe a protocol for collecting real clips with height ground truth.

1 Introduction

Computer vision for cue sports has largely been about finding balls on a plane: detect each ball in each frame, map it to table coordinates and track it [1]. That plane assumption breaks during some of the game's most consequential moments. A jump shot deliberately drives the cue ball into the slate so it rebounds over an obstructing ball; a hard break often lifts the cue ball and object balls off the cloth; and a jumped ball that clips the top of another ball can send either one airborne. Whether a ball left the cloth matters for rules (an illegal "scoop" under the ball versus a legal jump), for shot analytics and coaching, and for any system that reconstructs or simulates the game: a tracker that assumes planar motion silently misplaces an airborne ball and misreads apparent collisions.

This raises a simple question with no published answer: *from a single fixed camera, how reliably can we detect that a ball is airborne, and which visual evidence makes it possible?* The two camera placements that dominate practice behave very differently. An overhead table camera, the standard choice for streaming and analytics rigs, looks along the very axis in which the ball moves. An elevated broadcast camera behind the end rail sees height directly but foreshortens the table.

Contributions.

1. A billiards simulator with an airborne phase and a renderer that turns its output into video from both camera placements with exact per-frame height labels (Sec. 3). Existing open simulators keep every ball on the table [3].

2. A geometric analysis of the information each cue carries per millimetre of height, per camera and per image resolution (Sec. 4). It predicts which cues can work before any learning is involved.

3. A transparent detector built on pixel-level size, shadow and motion cues, evaluated per cue family and per camera, including both views fused (Secs. 5–7).

4. Seven hand-designed scenario clips that isolate the situations a detector must handle (Table 2), and a protocol for collecting real clips with height ground truth (Sec. 9).

2 Related work

Cue-sport vision and simulation. Ball detection and tracking for pool is well served by off-the-shelf object detectors [1], and analytic physics engines simulate shots precisely on the plane of the table [3, 6]. The physics of jump shots, including the role of the slate rebound, is discussed in the billiards literature [2, 7], but to our knowledge no vision work detects or measures airborne balls, and no public dataset labels ball height.

Monocular 3D ball trajectories in other sports. Recovering ball height from one camera is better studied in tennis and table tennis. Ponglertnapakorn and Suwajanakorn [8] lift 2D tennis tracks to 3D and stress that a 2D trajectory is consistent with many 3D motions. Kienzle et al. [4] train a network purely on physics-simulated trajectories and report transfer to real table-tennis broadcasts, and Kienzle et al. [5] build a robust two-stage pipeline for real footage. We follow the same synthetic-to-real philosophy, but pool differs in ways that change which cues work: the ball is small in the image, the camera is often directly above the action, flights are short (a few to a few dozen frames), and the lighting is a fixed set of lamps close to the table, which makes shadows unusually informative.

3 Simulation and rendering

Physics. Balls are rigid spheres ($R = 28.6\,\text{mm}$, $m = 170\,\text{g}$) on a 9-ft table, integrated at 2.4 kHz. On the cloth we use the standard sliding/rolling/spinning model (sliding friction $\mu_s = 0.2$, rolling resistance $\mu_r = 0.01$). Off the cloth a ball follows a ballistic path; air drag is negligible over sub-second flights. A landing applies a normal impulse with restitution $e_t = 0.5$ and a Coulomb friction impulse at the contact point; rebounds slower than $0.04\,\text{m/s}$ end the bounce sequence. Ball–ball collisions are resolved in 3D, so a high enough ball clears a blocker, and a low one strikes its upper hemisphere, driving it into the slate and making it hop. A jump shot is modelled the way it happens physically: the cue sends the ball *down* into the slate at the cue elevation, and the first landing impulse turns that into the jump. Cushions are walls of height 45 mm, and pockets swallow balls whose centres enter them. Table 1 lists the parameters; e_t is a little below the typical 0.6 quoted by Alciatore [2], and checking sensitivity to it is future work.

Parameter	Value	Parameter	Value
Ball radius / mass	28.575 mm / 170 g	Ball–ball restitution	0.95
Sliding / rolling friction	0.2 / 0.01	Ball–slate restitution e_t	0.5
Landing friction	0.2	Cushion restitution	0.8
Integration rate	2400 Hz	Video	60 fps, H.264
Lamps	3 area lights, 0.95 m up	Overhead camera	2.6 m, $f \approx 1090$ px

Table 1: Default simulation and rendering parameters. The randomised set varies camera placement, lamp height and spacing, sensor noise, shutter and exposure (Sec. 6).

Clip	What it tests	Cue apex
`break`	Full-rack break with a slightly elevated cue: cue ball and object balls hop off the cloth briefly.	9 mm
`elevated_draw`	Draw shot with an elevated cue (14 deg). Borderline: a ∼1 cm hop.	9 mm
`jump_clips_blocker`	Jump too low: cue ball strikes the top of the blocker, which hops. Positive, with a secondary object-ball hop.	75 mm
`jump_over_blocker`	Cue ball jumps a blocker 28 cm away and hits the 1-ball. Positive.	101 mm
`long_jump_off_axis`	Long, fast jump along the side rail, far from the overhead camera's optical axis (large parallax). Positive.	89 mm
`near_miss`	Cue ball passes 4 mm beside a ball without leaving the cloth: in 2-D the two balls almost touch. Hard negative.	1 mm
`stop_shot`	Level stun shot, cue ball stops dead. Negative.	0 mm

Table 2: The scenario clips (both camera views each). Apex is the cue ball's maximum height above the cloth.

Rendering. A ray caster renders the table (cloth, cushions, rails, pockets) and the balls (solid and striped textures with number discs, rotating with the simulated angular velocity, glossy highlights). The room has three hanging lamps, each an area light, that cast soft shadows on the cloth and on other balls. Each pixel takes four samples jittered within the pixel and within the shutter interval, which gives anti-aliasing and physically correct motion blur. Frames are tone-mapped, get Gaussian sensor noise, and are H.264-encoded at 60 fps. Two calibrated cameras are rendered: **overhead**, looking straight down from 2.6 m, and **broadcast**, behind the foot rail at a height of 1.55 m (Fig. 1). Both are 1280×720. For each clip we also store the empty-table frame a fixed camera can capture once.

Ground truth. For every frame and ball we store the centre at mid-exposure, the maximum height of the ball's lowest point above the cloth during the exposure, and whether that exceeds 1 mm.

Scenario clips. Table 2 lists the seven hand-designed clips. Each isolates one situation: clean positives, a hard negative where two balls almost touch in the image without either leaving the cloth, a borderline centimetre hop, a secondary object-ball hop, and the 16-ball break.

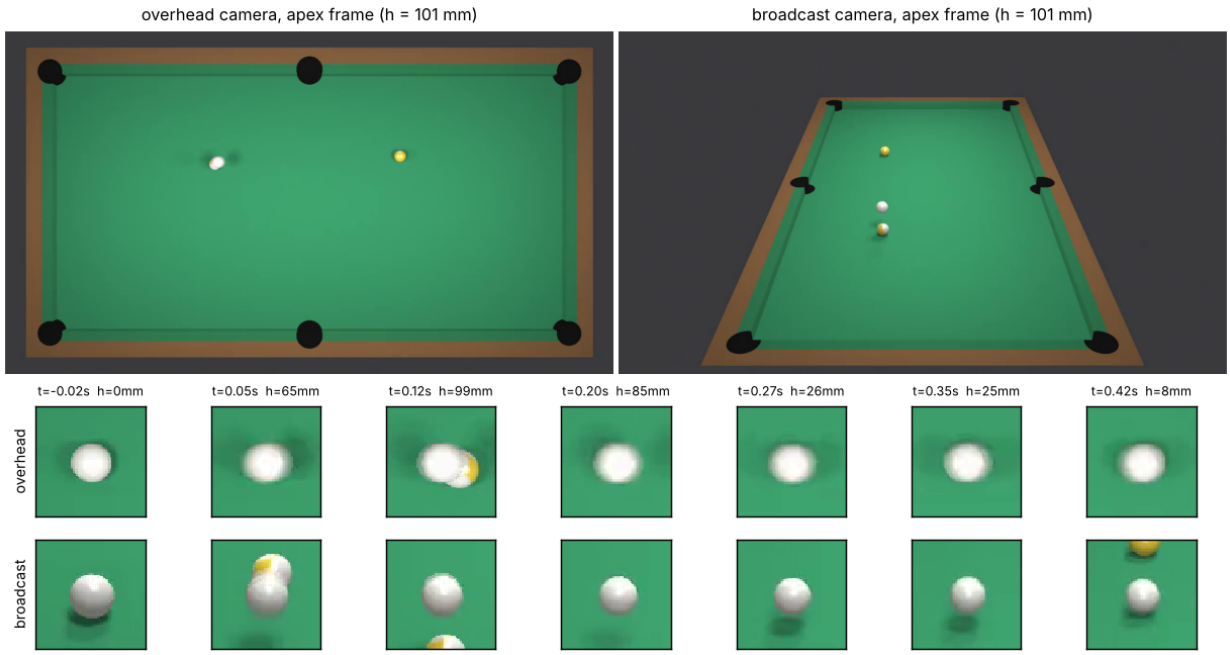


Figure 1: The `jump_over_blocker` clip at the apex of the jump, from the overhead (left) and broadcast (right) cameras, with zoomed crops across the flight. From overhead the airborne cue ball overlaps the blocker and its lamp shadows pull away from it; from the broadcast view the ball visibly separates from its shadow.

4 How much signal does each cue carry?

Before learning anything, we ask how many pixels each cue moves per millimetre of height, using the calibrated cameras alone (Fig. 2).

Size. Raising the ball by h brings it closer to an overhead camera at height H, scaling its image radius by $(H - R)/(H - R - h)$. At $H = 2.6$ m and 720p that is 0.05 px at $h = 10$ mm and only 0.49 px at 100 mm; even 4K gives 0.14 px per centimetre. The blur-robust radius estimator of Sec. 5 has a standard deviation of 0.96 px on on-cloth frames of the scenario clips (overhead, 720p; 1.7 px in the broadcast view). The geometric size signal of a 10 cm jump therefore sits at the noise floor, and that of a 1 cm hop is twenty times below it. The broadcast camera fares even worse (0.02 px), because height moves the ball across the line of sight rather than along it.

Parallax. An airborne ball appears displaced from where an on-cloth ball at the same table position would be. For the overhead camera the displacement is radial from the camera's nadir, so it is *zero* directly below the camera and 1.8 px per centimetre near a corner. For the broadcast camera it is 3.4 px per centimetre everywhere. A single frame cannot separate this displacement from a different table position, but a track can, because ballistic flight and rolling produce different image-space kinematics. That is what our motion cue tests.

Shadows. With lamps about a metre above the cloth, a ball's shadow from a lamp at horizontal offset d moves by roughly $d\,h/(L - h)$. From the lamp directly overhead, the shadow stays hidden under the ball, but the neighbouring lamps' shadows move by 3.8 px per centimetre at the table centre in the overhead view (5.0 px in broadcast). Shadows are therefore the strongest single-frame cue overhead, *provided the room has off-axis lamps*, as typical multi-shade fixtures do.

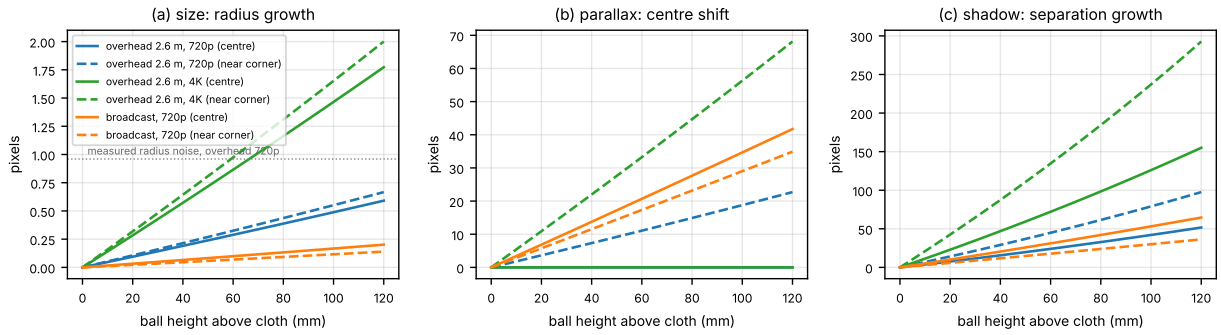


Figure 2: Pixels of signal per cue versus ball height, for a ball at the table centre (solid) and near a corner (dashed). Size changes are sub-pixel at any practical resolution; overhead parallax vanishes at the nadir; lamp shadows separate by pixels per centimetre.

5 Detector

The detector reads only what a deployed fixed camera has: the video, the empty-table frame, the camera calibration (in practice from the table corners) and the ball radius. We focus on the cue ball, which takes nearly all deliberate jumps.

Tracking and measurement. The cue ball is found in each frame as the white connected component nearest the predicted position. Its centre and radius are then refined using a soft coverage mask computed against the empty-table frame. Two choices proved essential (Sec. 8). First, coverage is measured as *loss of the cloth's green chromaticity* rather than as brightness above the cloth, so the ball's unlit underside counts and shadows on the cloth do not; this cut the median centre error in the broadcast view from 1.8 px to 0.29 px (overhead: 0.26 px). Second, the radius is taken from the coverage mask's *minor* second-moment axis, because motion blur only stretches the blob along the direction of travel. In the `jump_over_blocker` clip, the area-equivalent radius was 6–28% larger during flight, while the geometric change was at most 4%; area is a speed cue in disguise.

Cues. For each frame we compute three cue families. Each is normalised by the still frames before the strike, which absorbs per-installation bias. *Size*: the measured radius divided by the exact silhouette radius of an on-cloth ball at the back-projected position, with windowed mean and maximum. *Shadow*: inside a $4.5R$ disc around the ball, pixels darker than the empty table but with the same chromaticity are treated as shadow; we record shadow mass, centroid offset, spread and the shadow mass just outside the ball's rim. *Motion*: on three 7-frame windows (ending at, centred on and starting at the frame) we fit two physical models to the image track by least squares. A *rolling* model keeps the ball on the cloth, with deceleration along its path bounded by sliding friction and a small lateral term for swerve. A *ballistic* model has free initial height and vertical velocity under gravity, and constant horizontal velocity. The features are both residual costs, their log ratio and the fitted height, taken from the window that most favours flight, since a window that spans a bounce fits neither model. The ball's table position is added as context to every cue set. We deliberately *exclude* raw speed and blur length: in any finite shot generator they correlate with the label through the shot mix (jumps are struck hard), and a detector should not learn that prior.

	Overhead			Broadcast			Both views		
Cues	AP	Ev. rec.	FA/min	AP	Ev. rec.	FA/min	AP	Ev. rec.	FA/min
size	0.62	0.74	20.9	0.40	0.64	20.4	0.71	0.69	11.2
shadow	0.78	0.87	6.8	0.79	0.87	14.1	0.87	0.92	11.2
motion	0.59	1.00	24.3	0.86	0.77	8.3	0.91	0.87	11.2
size+shadow	0.81	0.90	15.6	0.79	0.87	11.2	0.87	0.85	8.3
size+motion	0.73	0.74	11.2	0.90	0.82	4.4	0.95	0.87	6.8
shadow+motion	0.85	0.92	13.6	0.92	0.85	9.2	0.96	0.90	5.8
all	0.86	0.82	12.6	0.93	0.79	3.4	0.96	0.90	4.9
Height MAE (mm)		14			14			11	

Table 3: Cue ablation on 60 held-out shots (39 jump events). AP: frame-level average precision. Ev. rec.: event recall. FA/min: false alarms per minute of video. Recall and false alarms trade off through the threshold chosen on the training shots, so compare cue strength by AP. Last row: mean absolute error of the regressed height on airborne frames.

Classifier. A gradient-boosted tree classifier maps the cue vector to $P(\text{airborne})$, smoothed with a 3-frame median; a regressor with the same features estimates the height.

6 Experiments

Data. We render 200 randomised shots (both views): half jump attempts with first-hop apex drawn from 1.5–14 cm and cue elevation from 25–55°, and half level shots with random speed, spin and a slightly elevated cue. A third of all shots place a ball on or beside the cue ball's path. Shots in which a ball ramps off another to more than 25 cm or leaves the table are resampled (about 5%). Per shot we randomise the overhead camera height (2.2–3.0 m), the broadcast camera placement, lamp height (0.8–1.1 m) and spacing (0.6–0.9 m), sensor noise, shutter (25–100% of the frame time) and exposure. Shots 0–139 train and 140–199 test; the scenario clips are never trained on.

Labels and metrics. A frame is positive if the cue ball is more than 5 mm off the cloth during the exposure. Frames with a 1–5 mm hop are "don't care" for frame metrics, because no camera here can resolve them (Sec. 4). We report frame-level average precision (AP), event recall, where an event is a flight with apex >5 mm, bounces are merged, and the event counts as detected if at least two of its frames fire, and false alarms per minute of video, i.e. predicted runs of at least two frames that touch no airborne frame. Thresholds are chosen by grouped 5-fold cross-validation on the training shots only.

7 Results

Which camera, which cue. With all cues, AP is 0.96 using both views (95% shot-level bootstrap interval 0.93–0.98), 0.93 from the broadcast camera (0.86–0.98) and 0.86 from overhead (0.78–0.91); Table 3. The single-cue rows follow the geometry of Sec. 4. From overhead the shadow is the best single cue (AP 0.78) and motion the weakest (0.59), as expected when parallax vanishes below the camera; the two are complementary (0.85 together). From the broadcast camera, motion is the best single cue (0.86), and the rolling-versus-ballistic log-ratio dominates the all-cue model's permutation importance. Size alone is the weakest cue in broadcast (0.40) but reaches 0.62 overhead, more than

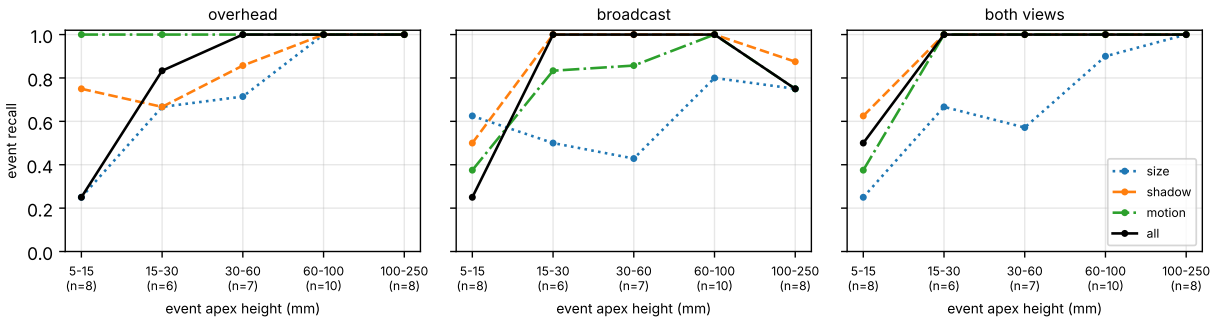


Figure 3: Event recall versus the apex height of the flight, per cue family and camera.

its sub-pixel geometric signal can explain. We attribute this to side effects that correlate with flight in our renderer, such as an airborne ball overlapping the ball it jumps and brightening as it nears the lamps. That is a hypothesis to test on real video, and we do not expect the effect to transfer. At the shot level (did the cue ball jump at all?), the maximum per-frame probability gives an AUROC of 0.87 overhead, 0.91 broadcast and 0.93 with both views.

Apex height decides detectability. Every one of the 25 test flights with an apex above 30 mm is detected by the overhead and both-view detectors, and 23 of 25 by the broadcast one (Fig. 3). Flights of 15–30 mm are mostly found (5–6 of 6). Hops of 5–15 mm are mostly missed (2 of 8 per single camera, 4 of 8 with both views); at those heights the signal is a few pixels at best. With only 39 test events, the event-recall intervals are wide: 0.69–0.93 overhead and 0.78–0.98 with both views.

False alarms come from collisions. The 60 test shots contain 2.1 minutes of video. The overhead all-cue detector raises 26 false alarms (0.43 per shot), the broadcast one 7 and the two-view one 10. Of the 26 overhead false alarms, 15 fall within 0.12 s of the cue ball striking another ball (9) or a cushion (6); 2 follow the cue strike, 2 sit just past a landing, outside the ±2-frame tolerance, and 7 occur during rolling. A collision is a velocity discontinuity that the rolling model cannot explain and that a ballistic fit absorbs more easily, and the other ball's shadow enters the shadow window at the same moment. The scenario clips show the same failure (Fig. 4): `stop_shot` fires in the broadcast view at contact, and `near_miss` fires (overhead and two-view) when the cue ball strikes the object ball, not as it grazes past its neighbour. A motion model with an explicit collision event is the obvious next step.

Height. The regressed height on airborne frames has a mean absolute error of 13.6 mm overhead, 14.2 mm broadcast and 10.7 mm with both views (median relative error 27%, 24% and 23%).

Scenario clips. The three clear jumps (apex 75–101 mm) are detected from every view, with $P(\text{airborne}) \approx 1$ through the first flight; overhead, the detection of `jump_over_blocker` drops out before the second bounce. The 9 mm hops in `break` and `elevated_draw` are missed except by the two-view detector on the break, as Fig. 3 predicts.

8 Lessons from building the detector

Three failure modes appeared before any learning, and all three would also afflict real footage. (i) *Brightness-based segmentation* misses the unlit underside of the ball in the broadcast view; the resulting radius estimate drifted by 12–20% as the ball rolled under different lamps, far more than the 4–7% that 10 cm of height produces. (ii) *Motion blur inflates area*: the fast airborne cue ball of `jump_over_blocker` looked 6–28% larger from overhead, while the true geometric change was at most 4%. A size cue measured by area is mostly a speed cue. (iii) *Unconstrained motion models explain flight*: a rolling model with free 2D acceleration fits the apparent ground track of a short flight about as well as a ballistic model. The model comparison only becomes informative once rolling deceleration is bounded by friction.

9 Towards real clips

The synthetic study fixes what to measure; the open question is transfer. We propose collecting real clips with height ground truth instead of scraping broadcast footage, which has none. A phone fixed overhead at about 2.5 m serves as the overhead view, a second camera behind the foot rail as the broadcast view, and a third camera at cloth level, perpendicular to the shot line, records 240 fps slow motion. That third view gives the ball's height directly against a printed scale on the rail, and a clap or LED flash synchronises the cameras. The shot list mirrors Table 2: jumps over blockers at several distances, jumps that clip the blocker, near-misses, elevated draw and follow shots, breaks, and level shots at all speeds. The protocol, labelling scheme and a clip checklist are in `data/real/README.md`.

10 Limitations

The ground truth is exact only for the simulated world. The physics omits cloth compression, ball–ball friction (throw) and the cue–ball–slate interaction during a jump, and e_t is a single constant. The renderer has no reflections, cloth texture, people or occlusion by players and cues, which a real overhead feed will contain. We detect the cue ball only; object-ball hops (present in two scenario clips) are left for future work. Most importantly, nothing here has been validated on real video yet.

11 Conclusion

Whether a fixed camera can tell that a pool ball has left the cloth depends on the camera and on which evidence it is asked to use. Apparent size, the intuitive cue for an overhead camera, carries a sub-pixel signal far below the measurement noise. Lamp shadows and a physically constrained comparison of rolling and ballistic motion carry the information instead. On synthetic video with exact ground truth, the broadcast placement detects jumps better than the overhead one, and both together are best (AP 0.96). Flights that rise above 3 cm are found reliably; centimetre hops and collision-induced false alarms remain open. The next step is the real-clip collection of Sec. 9, which will test whether these conclusions survive real cloth, real lamps and real players.

References

[1] Pool table analytics using object detection. Roboflow blog, `https://blog.roboflow.com/pool-table-analytics-object-detection`, 2023.

[2] David G. Alciatore. Technical proof B.10: draw shot cue elevation analysis. `https://billiards.colostate.edu/technical_proofs/new/TP_B-10.pdf`. Typical ball–table restitution 0.6, ball–cloth sliding friction 0.2.

[3] Evan Kiefl. Pooltool: A Python package for realistic billiards simulation. *Journal of Open Source Software*, 9(101):7301, 2024. doi: 10.21105/joss.07301.

[4] Daniel Kienzle, Robin Schön, Rainer Lienhart, and Shin'ichi Satoh. Towards ball spin and trajectory analysis in table tennis broadcast videos via physically grounded synthetic-to-real transfer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVSports)*, 2025. arXiv:2504.19863.

[5] Daniel Kienzle, Katja Ludwig, Julian Lorenz, Shin'ichi Satoh, and Rainer Lienhart. Uplifting table tennis: A robust, real-world application for 3D trajectory and spin estimation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2026. arXiv:2511.20250.

[6] Will Leckie and Michael Greenspan. An event-based pool physics simulator. In *Advances in Computer Games*, pages 247–262. Springer, 2006.

[7] Wayland C. Marlow. *The Physics of Pocket Billiards*. 1995.

[8] Puntawat Ponglertnapakorn and Supasorn Suwajanakorn. Where is the ball: 3D ball trajectory estimation from 2D monocular tracking. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVSports)*, 2025. arXiv:2506.05763.

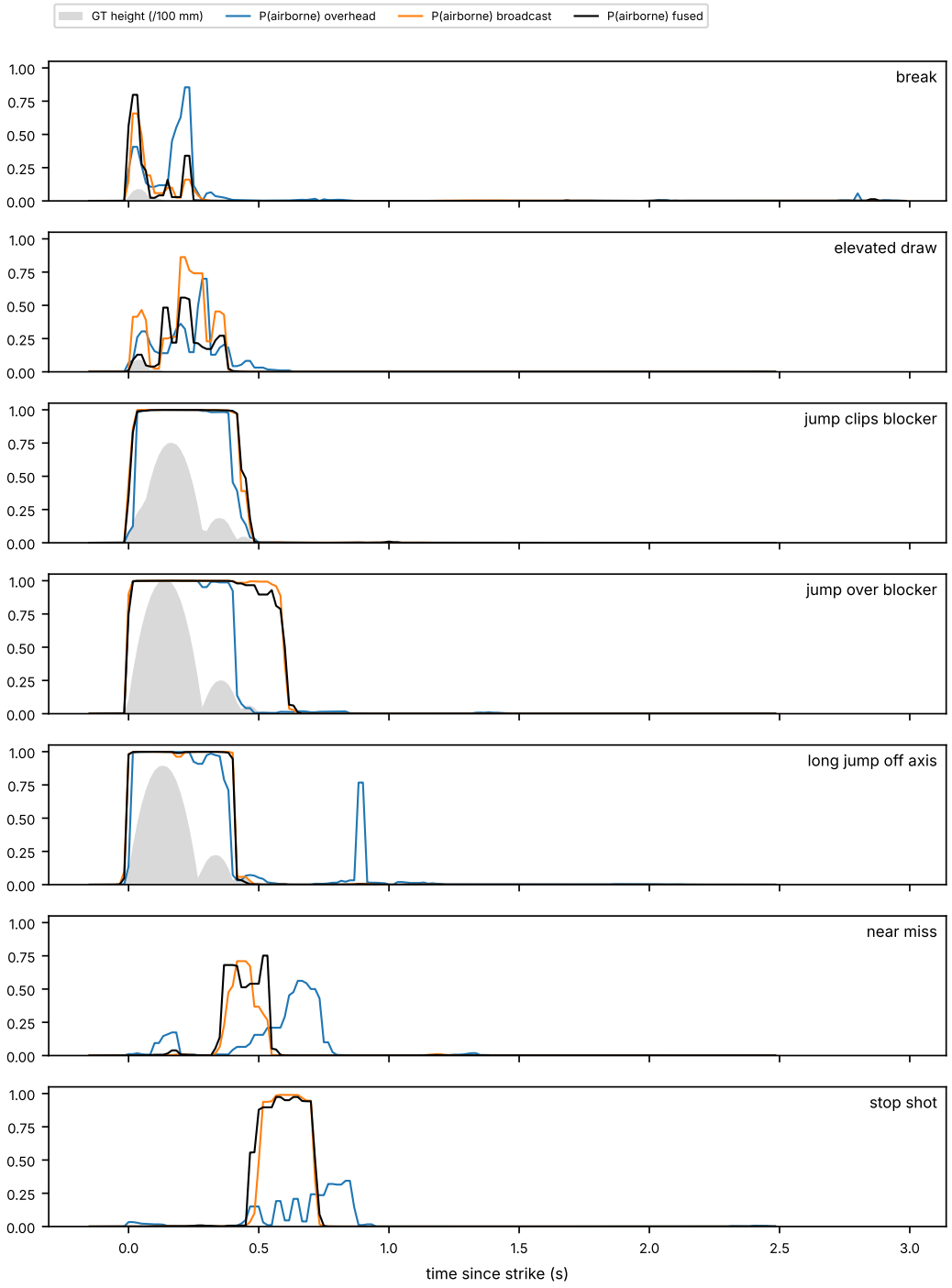


Figure 4: Scenario clips: predicted $P(\text{airborne})$ of the all-cue detectors (trained on random shots only) against the ground-truth cue-ball height (grey).